

浙江大学

项目开发计划书



项目主题： 校园学术活动智能推荐平台

任课老师： 邓水光

项目组员： 黄晗 宫上珺 孟祥烨

李妍雅 庞星磊 朱致远

2026 年 5 月 10 日

目录

第 1 章 项目概述	1
1.1 项目背景简述	1
第 2 章 开发范围	2
2.1 第一阶段开发范围	2
2.2 暂缓开发功能	2
2.3 最小可交付版本说明	3
第 3 章 总体开发方案	4
3.1 系统总体架构	5
3.2 前端开发方案	5
3.3 后端开发方案	5
3.4 数据库与爬虫开发方案	6
3.5 测试与文档开发方案	6
第 4 章 成员分工	8
4.1 团队角色划分	8
4.2 成员具体任务	8
第 5 章 任务分解	9
5.1 前端任务分解	9
5.2 后端任务分解	9
5.3 爬虫与数据库任务分解	10
5.4 测试与文档任务分解	10
5.5 任务依赖关系	10
第 6 章 进度安排	12
6.1 总体排期	12
6.2 第一阶段：项目初始化与基础链路	12

6.3	第二阶段：核心业务功能开发	13
6.4	第三阶段：联调与功能完善	13
6.5	第四阶段：测试、修复与交付准备	14
第 7 章	阶段性里程碑	15
第 8 章	协作方式	16
8.1	Git 协作规范	16
8.2	分支管理规范	17
8.3	文档同步规范	17
8.4	组员沟通联络	17
第 9 章	测试与验收计划	18
9.1	功能测试计划	18
9.2	接口测试计划	18
9.3	联调测试计划	18
9.4	验收演示流程	18
第 10 章	风险与应对措施	20
第 11 章	交付计划	21
11.1	代码交付	21
11.2	数据库交付	21
11.3	文档交付	21
11.4	演示交付	21

第 1 章 项目概述

1.1 项目背景简述

校园学术活动信息通常分散在学院官网、通知页面、公众号和群消息中，存在检索困难、更新不同步、时间冲突难判断等问题。校园学术活动智能推荐平台希望将活动聚合、筛选推荐、课表导入、冲突检测、加入日程和日历展示整合到一个统一入口，帮助师生更高效地安排学术活动。

第 2 章 开发范围

2.1 第一阶段开发范围

模块	第一阶段开发内容	预期效果
用户认证	简化登录、注册、Token 鉴权	用户可进入系统
活动浏览	活动列表、分页、详情页	用户可浏览活动信息
搜索筛选	关键词、类别、学院、校区、时间筛选	用户可快速定位活动
推荐模块	基于标签、热度、时间的规则推荐	首页展示推荐活动
课表导入	手动录入、CSV/Excel 导入, OCR 作为加分项	用户课程可进入日历
冲突检测	活动与课程/已有日程时间重叠判断	加入活动前提示冲突
日历展示	展示课程、活动、冲突、推荐状态	用户可查看个人安排
ICS 导出	导出活动或个人日程	可导入第三方日历
后台管理	活动新增、编辑、下架、爬虫数据修正	管理员可维护活动
爬虫数据	先采集一个学院的活动数据	提供真实活动样例

2.2 暂缓开发功能

暂缓功能	原因
统一身份认证 SSO/CAS	接入复杂，第一阶段使用本地账号替代
精确地图通勤测算	依赖外部 API，先预留字段
学术组队	与主流程关系较弱，后续扩展
笔记共享与评价	属于社交沉淀功能，第二阶段再做

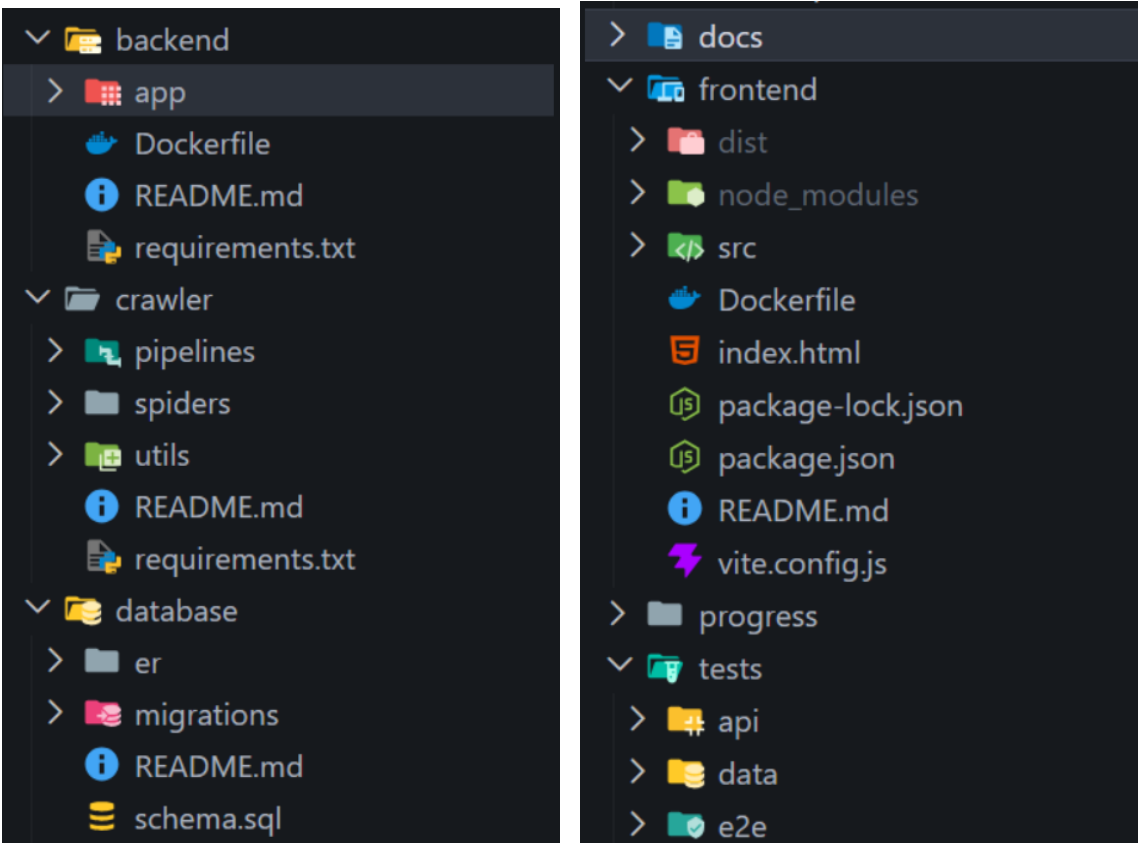
个人学术轨迹	依赖大量历史数据，第一阶段不优先
复杂 NLP 自动打标	先用人工标签或简单关键词规则
协同过滤推荐	第一阶段使用规则推荐即可

2.3 最小可交付版本说明

最小可交付版本至少应支持用户登录、活动列表展示、活动详情查看、关键词搜索、课表导入、加入日程、冲突检测、日历展示和 ICS 导出。即使爬虫、OCR 或推荐算法未完全完善，也必须通过样例数据和规则推荐保证完整演示流程可运行。

第 3 章 总体开发方案

目前我们其实已经初步搭建了项目框架，如下图所示：



SE-project/	
frontend/	# 前端项目，负责页面和交互
backend/	# 后端项目，负责 API 和业务逻辑
crawler/	# 爬虫脚本，负责采集学院活动数据
database/	# 数据库建表、种子数据、ER 图
docs/	# 项目文档、需求文档、接口规范、测试文档
tests/	# 自动化测试、接口测试、测试数据
progress/	# 成员个人开发日志和项目进度记录
README.md	# 项目总说明
docker-compose.yml	# 后期可选，一键启动服务

3.1 系统总体架构

系统采用前后端分离架构：前端负责页面展示和交互，后端负责业务逻辑、接口服务、权限控制、冲突检测和 ICS 文件生成，数据库负责存储用户、活动、课表、日程和标签数据，爬虫模块负责从学院站点采集活动信息并写入数据库。

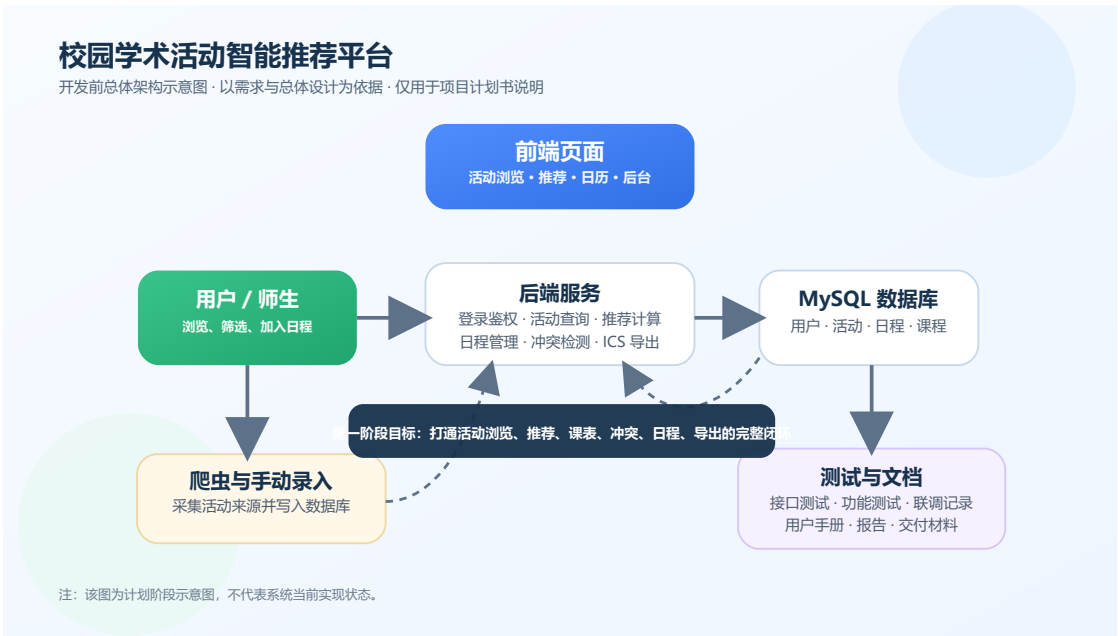


图 3.1 开发目标总体架构示意图

3.2 前端开发方案

前端以 Vue 3 + Vite + Element Plus 为基础，围绕“首页推荐、活动列表、活动详情、课表导入、个人日历、后台管理”六个核心页面展开开发。页面开发优先复用既定路由结构，先完成静态交互，再逐步替换为真实接口。

3.3 后端开发方案

后端采用 FastAPI 分层架构，路由层只负责参数接收与响应返回，业务逻辑集中在 services/，数据访问由 models/ 与数据库会话层完成。第一阶段优先实现登录注册、用户信息、活动列表、活动详情、推荐、日程、课表导入、后台管理和爬虫记录接口。

模块	主要职责
auth	登录、注册、Token 鉴权
users	用户信息、兴趣标签、个人中心

activities	活动列表、详情、搜索、筛选、排序
recommendations	规则推荐、推荐理由
schedules	日程查询、加入日程、冲突检测
courses	课表导入、课程保存、识别结果接收
admin	后台活动管理、爬虫记录、人工修正
crawler	爬虫任务触发、运行记录

3.4 数据库与爬虫开发方案

数据库以 database/schema.sql 为权威来源,核心表包括 user、activity、activity_tag、user_interest、course_schedule、schedule_event、registration、crawl_record、admin_log。表结构设计重点保证活动、标签、课程和日程之间的关联关系清晰，便于后续实现推荐与冲突检测。

爬虫部分先完成一个学院或一个数据源的采集闭环，重点实现字段抽取、时间清洗、重复数据去重、入库和运行日志记录。第一阶段以“能稳定产出样例活动数据”为目标，不追求全站覆盖。

3.5 测试与文档开发方案

测试采用“接口测试 + 手工功能测试 + 前后端联调测试 + 演示验收测试”的四层策略。测试负责人统一维护测试用例、Bug 记录和回归结果，确保每次接口变更都能同步到文档和测试清单中。

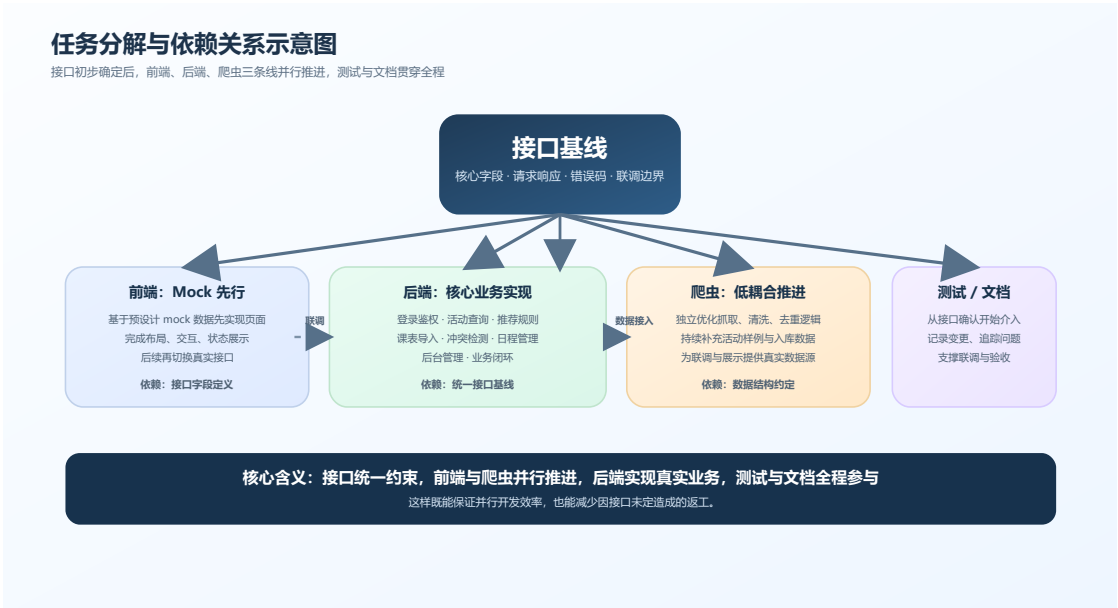


图 3.2 任务分解与依赖关系示意图

第 4 章 成员分工

4.1 团队角色划分

我们小组一共 6 名同学，采用的分工是：1 名前端、1 名爬虫与数据库、3 名后端、1 名测试与文书。

4.2 成员具体任务

成员	角色	主要负责内容	交付细则
朱致远	前端负责人	页面与交互开发	活动列表、活动详情、日历、课表导入、后台页面联调
宫上珺	爬虫与数据库负责人	数据来源与数据库设计	表结构、种子数据、活动样例数据、计院爬虫与入库
孟祥烨	后端负责人 1	后端基础与通用接口	数据库连接、ORM 基础、登录注册、用户信息、活动基础接口
李妍雅	后端负责人 2	日历与课表业务	课表导入、截图识别接口预留、冲突检测、日程、ICS 导出
庞星磊	后端负责人 3	推荐与筛选排序	活动搜索、标签/类别/校区筛选、排序、推荐规则、后台辅助统计
黄晗	测试与文书负责人	测试管理与项目文档	测试用例、接口测试、Bug 记录、进度记录、用户手册和报告材料

第 5 章 任务分解

5.1 前端任务分解

前端任务我们大致拟定按照 页面骨架 -> 接口接入 -> 交互完善 -> 演示优化的思路推进，预期的任务分解和产出如下表：

任务	产出
登录/注册页面	表单校验、登录态保存、路由跳转
首页推荐页	推荐活动卡片、热门活动入口、快捷导航
活动列表页	搜索框、筛选器、分页、排序、空状态
活动详情页	活动信息展示、标签、加入日程入口
日历页	月/周视图、颜色标识、冲突提示
课表导入页	文件上传、识别结果展示、确认入库
后台管理页	活动新增、编辑、下架、爬虫记录查看

5.2 后端任务分解

后端任务按数据层、业务层、接口层依次展开，先实现数据库查询和基础鉴权，再实现业务逻辑和管理接口。

任务	产出
数据库连接与 ORM	可复用的 DB 会话、模型和查询封装
登录注册与鉴权	/auth/login、Token、/users/me
活动基础接口	/activities、/activities/{id}
推荐规则接口	推荐列表、推荐理由、排序规则
日程与冲突检测	冲突判断、日程写入、日历查询
课表导入接口	课程保存、导入结果、OCR 预留
后台与爬虫接口	活动管理、爬虫运行、爬虫记录

5.3 爬虫与数据库任务分解

爬虫与数据库任务的核心目标是建立稳定的数据底座，确保前后端联调时有可用样例数据。

任务	产出
表结构复核	校验字段命名、外键、索引和默认值
种子数据准备	测试账号、测试活动、测试课程样例
爬虫字段抽取	标题、时间、地点、学院、标签、来源链接
数据清洗去重	去除重复活动、统一时间格式、清洗空字段
入库与日志	活动入库、抓取记录、失败信息记录

5.4 测试与文档任务分解

任务	产出
测试用例设计	按模块拆分的测试用例表
接口测试	健康检查、登录、活动列表、日程、管理接口
联调测试	前后端字段对齐、错误处理、边界条件修复
文档维护	进度记录、测试报告、用户手册、接口文档同步

5.5 任务依赖关系

简单来说，在接口初步确定好之后，前端负责的同学可以先利用预先设计的一版 mock 数据进行前端页面实现，而爬虫负责的同学的同样可以相对低耦合的去实现和优化爬取数据的处理逻辑。后端的同学则着手实现核心业务。测试和文档维护则贯穿开发始终。

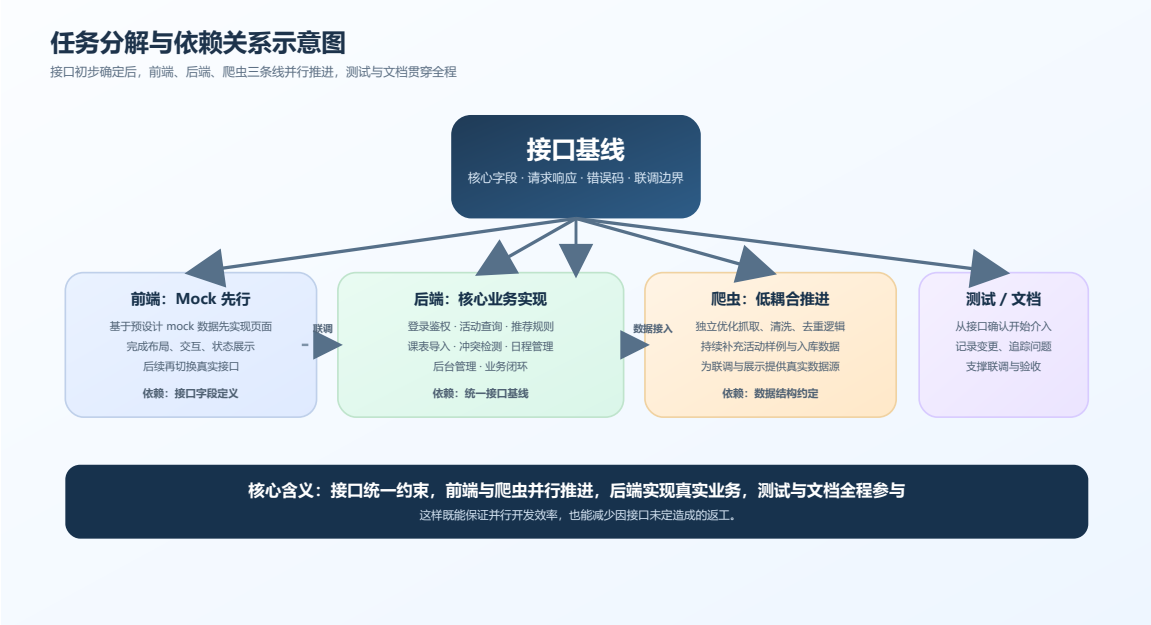


图 5.1 任务分解与依赖关系示意图

第 6 章 进度安排

6.1 总体排期

阶段	拟定周期	核心目标	主要产出
第一阶段	4-5 天	数据与基础接口优先	稳定表结构、登录、活动列表、详情接口
第二阶段	3-5 天	主流程联调	搜索筛选、推荐、课表导入、冲突检测、日程写入
第三阶段	4-5 天	演示闭环补齐	ICS 导出、后台管理、页面细节、异常处理
第四阶段	2-3 天	测试与交付准备	测试报告、用户手册、答辩材料、Bug 收敛

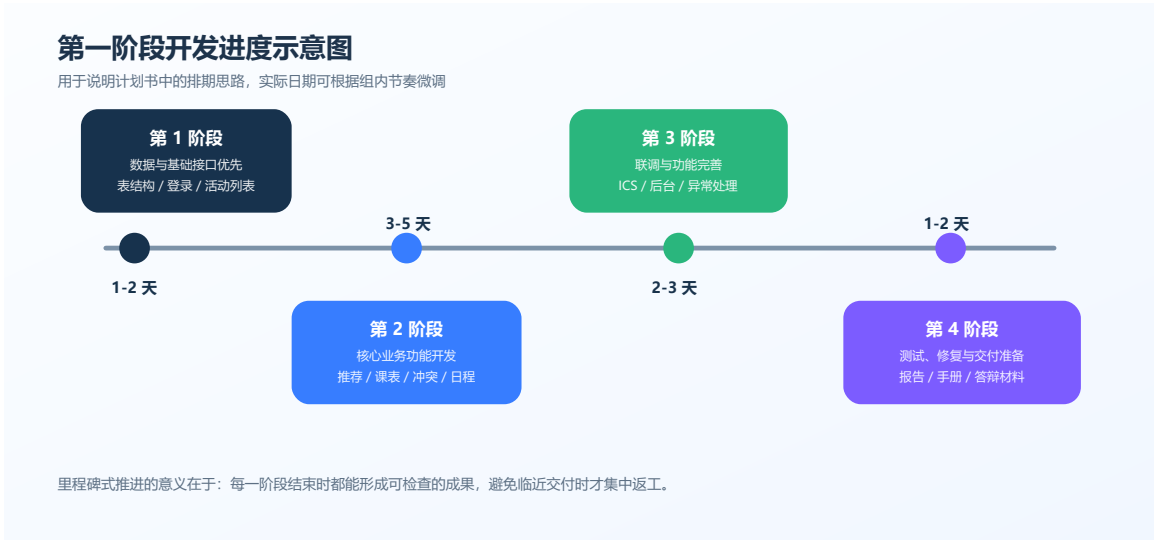


图 6.1 第一阶段开发进度示意图

6.2 第一阶段：项目初始化与基础链路

成员	任务
----	----

gsj	稳定数据库表结构，补充种子数据，整理字段说明
mxy	完成数据库连接、ORM 基础、用户登录、用户信息、活动列表和详情接口
hh	建立测试用例骨架，记录字段和接口变更
zzy	基于页面骨架完成活动列表、详情、登录页的联调准备
pxl	确认筛选和推荐所需字段，给出推荐分第一版规则
lyy	确认日程和课程表字段，设计冲突检测输入输出

阶段完成标志：前端能调用真实活动列表接口，后端能从 MySQL 返回活动数据，测试同学能跑通健康检查、登录和活动列表。

6.3 第二阶段：核心业务功能开发

成员	任务
zzy	对接登录、活动列表、活动详情、推荐活动、日历事件接口
gsj	完成第一版爬虫数据解析，准备活动样例数据入库
mxy	完成注册、JWT 鉴权依赖、后台活动新增/编辑/下架基础接口
lyy	完成课程录入、CSV/Excel 导入、日程查询、冲突检测、加入日程
pxl	完成关键词搜索、类别/标签/校区筛选、热度/时间/推荐分排序
hh	开始执行接口测试和前后端联调测试，记录 Bug 和复现步骤

阶段完成标志：用户可以登录，可以浏览和筛选活动，可以查看活动详情，可以导入或录入课程，可以检测活动与课程冲突，可以把活动加入日程并在日历中看到。

6.4 第三阶段：联调与功能完善

成员	任务
zzy	完成页面交互细节、状态标签、错误提示和演示截图
gsj	修正爬虫清洗和去重，保证演示数据质量
mxy	完善错误响应、权限判断、管理员接口稳定性

lyy	完成 ICS 导出，OCR 接口可根据时间做预留或简化演示
pxl	调整推荐理由和推荐排序，补充后台统计接口
hh	完成第一轮系统测试、测试报告初稿、用户手册初稿

阶段完成标志：答辩演示主流程可以连续跑通，测试报告有执行记录，用户手册能指导普通用户和管理员完成核心操作。

6.5 第四阶段：测试、修复与交付准备

成员	任务
全体	修复高优先级 Bug，统一演示环境，确认代码合并状态
zzy	准备前端展示截图和演示流程说明
gsj	准备数据库 ER 图、活动数据来源说明、爬虫运行截图
mxy、lyy、pxl	准备后端接口、核心逻辑和 Swagger 截图
hh	汇总测试报告、用户手册、进度文档和答辩材料

第 7 章 阶段性里程碑

这部分其实就对应了上一节的分阶段进度安排，这里不过多赘述，仅仅简要阐释完成标准。

里程碑	内容	完成标准
一	项目骨架与数据库完成	数据库建表、测试数据、后端基础配置和前端路由骨架全部可用
二	活动浏览主链路完成	活动列表、详情页、分页、搜索和基础推荐可联调
三	日历与冲突检测完成	课表导入、日程展示、冲突提示和加入日程可以闭环
四	完整演示流程完成	从登录到导出 ICS 的主流程可连续演示，无阻断问题
五	最终交付版本完成	代码、数据库、文档、测试报告和答辩材料统一归档，可提交

第 8 章 协作方式

8.1 Git 协作规范

项目采用 main 稳定主分支，开发时按模块建立功能分支，暂定规范如下：

main

frontend/xxx

backend/xxx

crawler/xxx

database/xxx

docs/xxx

test/xxx

fix/xxx

稳定主分支

前端功能

后端功能

爬虫功能

数据库相关

文档相关

测试相关

Bug 修复



图 8.1 Git 协作流程示意图

同时我们规定的 commit 提交的信息规范，方便组员之间对齐进度，知道对方现在做到哪里了：

类型	含义	示例
feat	新增功能	feat: add activity list page
fix	修复 Bug	fix: fix schedule conflict check

doc	文档修改	docs: update environment setup guide
style	样式调整, 不影响逻辑	style: polish login page layout
refactor	重构代码, 不新增功能	refactor: split activity service
test	测试相关	test: add auth api tests
chore	配置、依赖、脚手架等杂项	chore: update gitignore
build	构建或打包配置	build: add frontend dockerfile
ci	CI/CD 配置	ci: add github actions workflow

8.2 分支管理规范

1. 每个功能尽量只改自己的模块, 避免无关文件互相覆盖。
2. 合并前必须完成本地测试或基础构建验证。
3. 与接口相关的变更必须同步通知前端、测试和文档负责人。

8.3 文档同步规范

开发过程中的关键文档包括 docs/api-contract.md、docs/test-plan.md、docs/test-cases.md、docs/user-manual.md 和 progress/ 下的进度记录。所有影响用户操作或接口结构的改动, 都应在对应文档中留下记录。

8.4 组员沟通联络

我们按照任务进度定期线上会议对进度, 借助腾讯会议并且回放留档。在日常开发过程中我们则通过微信群直接联络, 及时 update 个人任务。GitHub 上的项目 docs/ 目录下面用于记录项目相关的各种文档及规范, 直接替代共享文档。

第 9 章 测试与验收计划

9.1 功能测试计划

测试模块	重点检查内容
登录注册	正常登录、错误密码、Token 失效
活动列表	分页、筛选、排序、空状态
活动详情	数据完整性、标签展示、加入入口
课表导入	文件格式、导入结果、错误提示
冲突检测	与课程冲突、与已有日程冲突
日程与日历	事件展示、颜色标识、状态标签
后台管理	新增、编辑、下架、记录查看

9.2 接口测试计划

接口测试优先覆盖健康检查、登录、活动列表、活动详情、推荐、日程、课表导入、后台活动管理和爬虫记录接口。接口测试结果应保留请求参数、返回内容和执行时间，便于回归对比。

9.3 联调测试计划

联调阶段重点检查前后端字段是否一致、空数据是否能正确展示、异常返回是否能被前端友好处理、冲突弹窗是否能正确触发。

9.4 验收演示流程

目前拟定的演示流程为：

1. 登录系统。
2. 进入首页查看推荐。
3. 打开活动列表并进行筛选。

4. 查看活动详情。
5. 导入课表并生成日程。
6. 点击加入活动，触发冲突检测。
7. 在日历中查看活动与课程。
8. 导出 ICS 文件。
9. 进入后台维护活动并查看爬虫记录。

第 10 章 风险与应对措施

风险类型	具体表现	应对措施
技术风险	后端接口较多、实现进度慢	优先实现主流程接口，先保证可联调
进度风险	某一模块拖慢整体集成	按里程碑推进，设置交接节点
数据风险	样例活动过少或字段不统一	提前冻结字段，补充种子数据
联调风险	前后端字段不一致	接口先行，变更同步到文档与测试
展示风险	演示时数据不足或流程中断	提前准备演示数据和备用流程

第 11 章 交付计划

11.1 代码交付

交付内容包括前端源码、后端源码、爬虫代码、数据库脚本和测试代码。代码应保证可以通过项目现有启动方式运行。

11.2 数据库交付

交付内容包括 `database/schema.sql`、`database/seed.sql` 和必要的字段说明，确保他人可重新初始化数据库并复现实验环境。

11.3 文档交付

交付文档包括开发计划书、总体设计报告、接口文档、测试计划、测试用例、测试报告、用户手册和进度记录。

11.4 演示交付

演示交付要求准备稳定的演示数据、可访问的接口、可展示的截图和可重复的主流程脚本，保证答辩现场不依赖临时修改。